Image Filtering and Image Enhancement

Complete University Notes — Computer Vision (B.Tech CSE)

> **How to use this document:** Read chapters in order the first time. Before your exam, re-read only the **"Quick Revision Sheet"** at the end of each chapter and the **Master Cheat Sheet** at the very end.

Table of Contents

1. Introduction to Image Filtering
2. The Kernel (Filter Mask) and Convolution Mechanics
3. Convolution vs. Correlation
4. Spatial vs. Frequency Domain Filtering
5. Smoothing (Low-Pass) Filters
6. Sharpening (High-Pass) Filters
7. Edge Detection Fundamentals
8. Sobel Operator
9. Prewitt Operator
10. Roberts Operator
11. Laplacian Edge Detection
12. Canny Edge Detector
13. Image Noise Models
14. Noise Reduction Techniques
15. Frequency Domain Filtering (Fourier-based)
16. Morphological Operations
17. Orthogonal Transforms (DCT, Haar, Walsh)
18. Master Comparison Tables
19. OpenCV Practical Program Bank
20. Solved Numerical Examples (Convolution)
21. University Exam Question Bank (2/5/10 marks, Viva, MCQ)
22. Master Cheat Sheet (1 page)

Chapter 1: Introduction to Image Filtering

1.1 What Is Image Filtering?

> **Definition:** Image filtering is the process of modifying or enhancing an image by applying a mathematical operation to each pixel using information from its **neighboring pixels**. The operation is usually performed using a small matrix called a **kernel** (or mask/filter) that slides over the image.

Think of a digital image as a grid of numbers (pixel intensities, 0–255 for 8-bit grayscale). Filtering means: **replace every pixel's value with some function of the values in its neighborhood.**

```
Original Image (numbers)          Filtered Image (new numbers)
 12  15  20  18                    14  16  18  17
 10  14  22  19        --->        13  17  19  18
 11  13  25  21                    12  18  20  19
  9  12  19  20                    11  15  17  18
```

Intuition: Imagine every pixel "asking its neighbors for advice" before deciding its own final value. Depending on *how* it asks (averages? picks the median? looks for sudden jumps?), we get very different effects — blurring, sharpening, edge highlighting, noise removal, etc.

1.2 Why Is Filtering Needed?

Raw images captured by cameras/sensors are rarely perfect. Filtering is needed to:

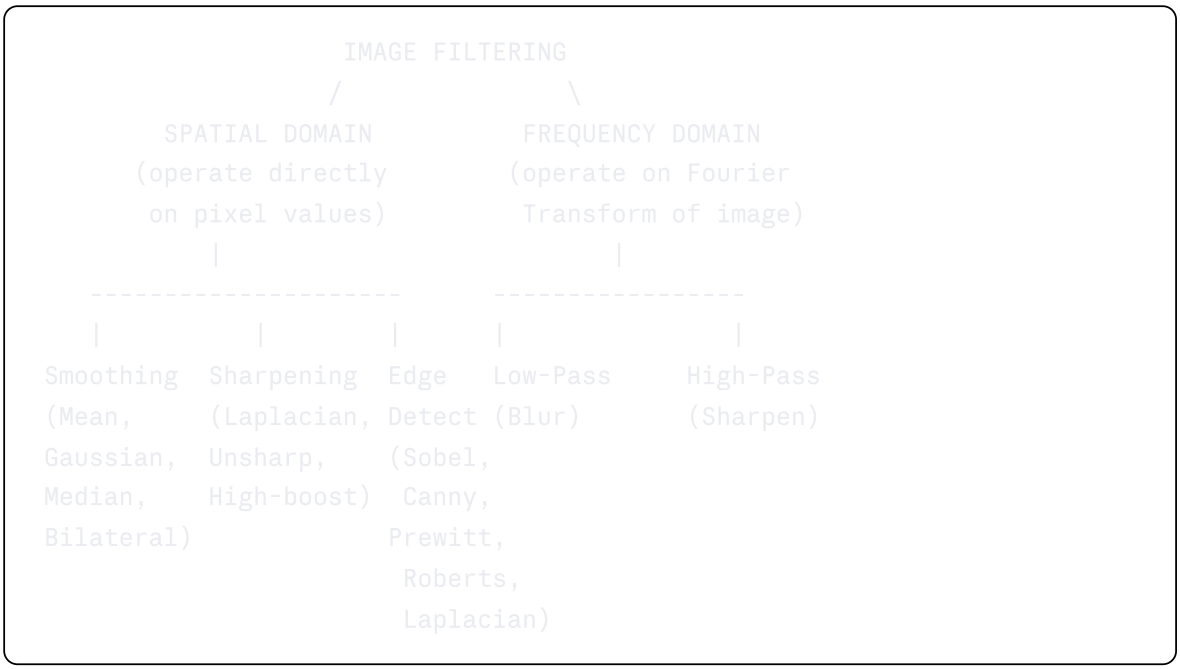
1. **Remove noise** introduced during image acquisition or transmission (sensor noise, low light, transmission errors).
2. **Enhance features** such as edges, corners, and textures for further processing (object detection, recognition).
3. **Smooth/blur** images intentionally (privacy blur, background blur, pre-processing before edge detection to avoid false edges from noise).
4. **Sharpen** images to recover detail lost due to blurring (out-of-focus correction, medical image clarity).
5. **Prepare images for higher-level Computer Vision tasks** — segmentation, feature extraction, object detection, OCR, etc. all perform much better on a properly pre-processed image.
6. **Compression** — some filters (frequency domain / transforms) help represent images with fewer numbers.

> 📌 **Note Box — Filtering vs. Enhancement**
> - **Image Filtering** = a general mathematical operation using neighborhood information (can smooth, sharpen, detect edges, remove noise).
> - **Image Enhancement** = the *goal* of improving visual quality/perceptibility of an image (contrast stretch, histogram equalization, sharpening, noise removal) — filtering is one of the **tools** used to achieve enhancement.
>
> In short: **All enhancement uses filtering, but not all filtering is for enhancement** (e.g., edge detection filters are for feature extraction, not visual "beautification").

1.3 Real-World Applications

Application Domain	Use of Filtering
Medical Imaging (MRI, X-ray, CT)	Noise removal, edge enhancement for tumor/tissue boundary detection
Satellite/Remote Sensing	Noise removal, sharpening for land-use classification
Autonomous Vehicles	Edge/lane detection, noise removal from camera feed
Face Recognition / Biometrics	Smoothing, sharpening, noise removal before feature extraction
Document Scanning / OCR	Binarization, noise removal, morphological cleanup
Photography / Social Media filters	Smoothing (skin), sharpening, bokeh (blur)
Video Surveillance	Denoising in low-light, motion blur reduction
Industrial Inspection	Edge detection for defect/crack detection
Astronomy	Noise removal from telescope images, contrast enhancement

1.4 Categories of Filtering (Preview)



We will study **spatial domain filtering in depth first** (Chapters 2–14), then **frequency domain filtering** (Chapter 15), then **morphological** and **transform-based** methods (Chapters 16–17).

1.5 Common Mistakes Students Make

- ❌ Confusing "filtering" with only "blurring" — filtering is a general term; blurring is just one type.
- ❌ Thinking filtering only works on grayscale — it works on color images too (apply the same kernel to each channel R, G, B separately, or convert to a suitable color space).

- ❌ Forgetting that filter output values may go out of the [0, 255] range or become negative — need **clipping/normalization**.
- ❌ Believing bigger kernel = always better — bigger kernels increase blur/computation but may destroy fine detail.

1.6 Chapter Summary

> Image filtering modifies each pixel using neighborhood information via a sliding kernel. It is essential for **noise removal, enhancement, feature extraction, and pre-processing** in nearly every computer vision pipeline. Filtering splits broadly into **spatial domain** (direct pixel operations) and **frequency domain** (operations on the Fourier Transform).

📋 **Quick Revision — Chapter 1**

- Filtering = modifying pixel using neighborhood + kernel.
- Needed for: noise removal, enhancement, feature extraction, pre-processing, compression.
- Two broad types: Spatial domain, Frequency domain.
- Enhancement is a *goal*; filtering is the *tool*.
- Applications: medical imaging, satellite imaging, autonomous vehicles, OCR, face recognition.

Chapter 2: The Kernel (Filter Mask) and Convolution Mechanics

2.1 Definition of a Kernel

> **Definition:** A **kernel** (also called a **mask**, **filter**, or **template**) is a small matrix of numbers, usually of odd size (3×3, 5×5, 7×7), which is used to perform image filtering. Each entry in the kernel is called a **weight** or **coefficient**.

Example — a simple 3×3 averaging kernel:

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Why odd size (3×3, 5×5...)? So that there is a well-defined **center pixel** — the pixel being replaced/computed sits exactly in the middle, with an equal number of neighbors on every side.

2.2 The Sliding Window Concept

The kernel is placed over the top-left corner of the image, the sum-of-products (or another operation) is computed, and the result becomes the new value of the pixel *under the kernel's center*. The kernel then **slides** one step (stride) to the right, and the process repeats — across the whole image, row by row.

```
Image (I):                    Kernel (K), 3x3
+----+----+----+----+----+
| 10 | 20 | 30 | 40 | 50 |        +---+---+---+
+----+----+----+----+----+        | a | b | c |
| 15 | 25 | 35 | 45 | 55 |        +---+---+---+
+----+----+----+----+----+        | d | e | f |
| 12 | 22 | 32 | 42 | 52 |        +---+---+---+
+----+----+----+----+----+        | g | h | i |
| 18 | 28 | 38 | 48 | 58 |        +---+---+---+
+----+----+----+----+----+

Step 1: Place K center on I(row2,col2)=25
        Window = [10 20 30; 15 25 35; 12 22 32]
        Output(2,2) = sum(window .* K)

Step 2: Slide K one column to the right (stride = 1)
        Window = [20 30 40; 25 35 45; 22 32 42]
        Output(2,3) = sum(window .* K)
        ... and so on until the kernel reaches the right edge,
        then it moves down one row and starts again from the left.
```

Intuition: It's exactly like a photocopier scanning head or a "magnifying glass" moving across the image, looking at a small neighborhood at a time and writing down a new number for the pixel in the center of that neighborhood.

2.3 Padding

Problem: When the kernel center is placed near the border of the image, part of the kernel "hangs off" the edge — there is no pixel data there!

```
       Kernel hanging off the left edge:
       ?   ?   ?
       ?  [X] 30      <- X is near the border, kernel needs
       ?   ?   ?          pixels to the LEFT that don't exist
```

> **Definition — Padding:** Padding means artificially adding extra rows/columns of pixels around the border of the image so that the kernel has valid data to operate on, even at the edges.

Types of padding:

Padding Type	Description	Example (1D border)
Zero padding	Add pixels with value 0	`0 0
Replicate (Clamp/Edge)	Repeat the border pixel value	`10 10
Reflect (Mirror)	Mirror the image content across the border	`30 20
Wrap (Circular)	Wrap around to the opposite edge	`20 30
Valid / No padding	Don't pad — output image becomes smaller	Output size shrinks

Effect of padding on output size:

$$\text{Output size} = \frac{N - K + 2P}{S} + 1$$

Where N = input size, K = kernel size, P = padding, S = stride.

- If $P = \frac{K-1}{2}$ (called **"same" padding**), output size = input size.
- If $P = 0$ (called **"valid"**), output is smaller than input by $(K - 1)$ in each dimension.

2.4 Stride

> **Definition:** Stride is the number of pixels the kernel moves (jumps) after each operation.

- **Stride = 1** → kernel moves 1 pixel at a time → dense, high-resolution output (standard for image filtering).
- **Stride = 2** → kernel skips every alternate position → output is roughly half the size (used in CNN pooling/downsampling, less common in classical filtering where we usually want stride = 1 to preserve resolution).

```
Stride = 1:  [K][K][K][K]...      (overlapping heavily, output ≈ input
size)
Stride = 2:  [K]  [K]  [K] ...     (less overlap, smaller output)
```

2.5 Worked Numerical Example — 3×3 Kernel Convolution

Given image patch (5×5), and a 3×3 averaging kernel, stride = 1, no padding ("valid"):

$$I = \begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 4 & 5 & 6 & 1 & 2 \\ 7 & 8 & 9 & 2 & 3 \\ 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 0 \end{bmatrix} \qquad K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Step 1: Output size = $(5 - 3 + 0)/1 + 1 = 3$. So output will be a 3×3 matrix.

Step 2: Compute output(1,1) — place the kernel over the top-left 3×3 block:

$$\text{block} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$\text{sum} = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$$

$$\text{output}(1,1) = 45 \times \frac{1}{9} = 5.0$$

Step 3: Slide right by 1 (stride 1) — new block:

$$\begin{bmatrix} 2 & 3 & 0 \\ 5 & 6 & 1 \\ 8 & 9 & 2 \end{bmatrix} \Rightarrow \text{sum} = 36 \Rightarrow \text{output}(1,2) = 4.0$$

Step 4: Continue sliding — final output (you should verify these by hand for practice):

$$\text{Output} \approx \begin{bmatrix} 5.0 & 4.0 & 2.44 \\ 4.67 & 4.44 & 3.78 \\ 4.22 & 4.89 & 4.89 \end{bmatrix}$$

Exam Tip: In university exams, you are almost always asked to do this manually for one or two positions — always show the sliding block, the element-wise multiplication, and the final sum.

2.6 Common Mistakes

- ❌ Forgetting to **flip the kernel** for true convolution (see Chapter 3) — most exam numericals actually use *correlation* even when they say "convolution"; always check whether the kernel is symmetric (if symmetric, flipping doesn't matter!).
- ❌ Wrong output size calculation — always use the formula, don't guess.
- ❌ Forgetting to normalize (divide by sum of kernel weights) when the kernel weights don't already sum to 1.
- ❌ Applying padding incorrectly (padding with wrong values changes border results significantly).

2.7 Chapter Summary

> A **kernel** is a small matrix of weights. Filtering slides this kernel over the image (**sliding window**), computing a weighted operation at each position. **Padding** handles border pixels; **stride** controls how far the kernel jumps each step. Output size = $\frac{N-K+2P}{S} + 1$.

📋 **Quick Revision — Chapter 2**

- Kernel = small matrix (usually odd size) of weights.
- Sliding window: kernel moves across the image, computing new pixel values.
- Padding types: Zero, Replicate, Reflect, Wrap, Valid (none).
- "Same" padding → output size = input size. "Valid" → output shrinks.
- Stride = step size of kernel movement (usually 1 in filtering).
- Output size formula: $(N - K + 2P)/S + 1$.

Chapter 3: Convolution vs. Correlation

3.1 Definition of Convolution

> **Definition (2D Convolution):** Convolution of image I with kernel K is defined as: > $$ > (I * K)(x,y) = \sum_{i=-a}^{a}\sum_{j=-b}^{b} I(x-i, y-j)\, K(i,j) > $$ > The key operation is that the **kernel is flipped (rotated 180°)** both horizontally and vertically before being applied.

3.2 Definition of Correlation

> **Definition (2D Cross-Correlation):** Correlation is the same sliding-window, weighted-sum operation, **but without flipping the kernel**: $$ (I \star K)(x,y) = \sum_{i=-a}^{a}\sum_{j=-b}^{b} I(x+i, y+j), K(i,j) $$

Intuition: Convolution flips the kernel 180° before sliding; correlation does not. If the kernel is **symmetric** (like the Mean/Gaussian kernel), flipping makes no difference — correlation and convolution give identical results! If the kernel is **asymmetric** (like Sobel), the results differ.

3.3 Visual Difference

```
Kernel K:                Flipped Kernel (180° rotation) used in TRUE
convolution:
  1  2  3                    9  8  7
  4  5  6        ---->       6  5  4
  7  8  9                    3  2  1
```

3.4 Step-by-Step Example

Given a 1D signal $I = [1, 2, 3, 4, 5]$ and kernel $K = [1, 0, -1]$ (a simple edge-detecting kernel), find output at position 3 (center = index 3, value 3), using a 3-tap window $[2, 3, 4]$.

Correlation (no flip):

$$\text{output} = (2 \times 1) + (3 \times 0) + (4 \times -1) = 2 + 0 - 4 = -2$$

Convolution (flip kernel first): Flipped kernel = $[-1, 0, 1]$

$$\text{output} = (2 \times -1) + (3 \times 0) + (4 \times 1) = -2 + 0 + 4 = 2$$

Notice the sign flips! This matters for directional filters like Sobel/Prewitt.

3.5 Why Does It Matter in Practice?

Aspect	Convolution	Correlation
Kernel handling	Flipped 180°	Not flipped
Symmetric kernels (Mean, Gaussian, Laplacian)	Same result as correlation	Same result as convolution
Asymmetric kernels (Sobel, Prewitt, custom directional)	Different result (sign/direction may change)	Different result
Used in classical image processing textbooks	"Convolution" often loosely means correlation in practice	OpenCV's `filter2D()` actually performs **correlation** by default!
Mathematical properties	Commutative, Associative	Not commutative

> 📌 **Important Exam Point:** OpenCV's `cv2.filter2D()` performs **correlation**, not true mathematical convolution. To get true convolution in OpenCV, you must flip the kernel manually before calling `filter2D()`.

3.6 Common Mistakes

- ❌ Assuming convolution and correlation always give the same result — **only true for symmetric kernels**.
- ❌ Not flipping the kernel when a question explicitly asks for "convolution" with an asymmetric kernel.
- ❌ Confusing convolution (signal processing definition) with the "convolution" used loosely in CNN literature (which is technically correlation in most deep learning frameworks too!).

3.7 Chapter Summary

> Convolution flips the kernel 180° before the weighted sliding-sum; correlation does not. For symmetric kernels (Mean, Gaussian) both are identical; for asymmetric kernels (Sobel, Prewitt) results differ. OpenCV's `filter2D` performs correlation by default.

📋 **Quick Revision — Chapter 3**

- Convolution = flip kernel + slide + weighted sum.
- Correlation = slide + weighted sum (no flip).
- Symmetric kernel ⇒ convolution = correlation.
- OpenCV `filter2D()` = correlation (not true convolution).
- CNNs also technically use correlation, called "convolution" by convention.

Chapter 4: Spatial vs. Frequency Domain Filtering

4.1 Spatial Domain Filtering

> **Definition:** Operating **directly on pixel values** of the image, using neighborhood operations (kernels/masks), as covered in Chapters 2–3.

$$g(x, y) = T[f(x, y)]$$

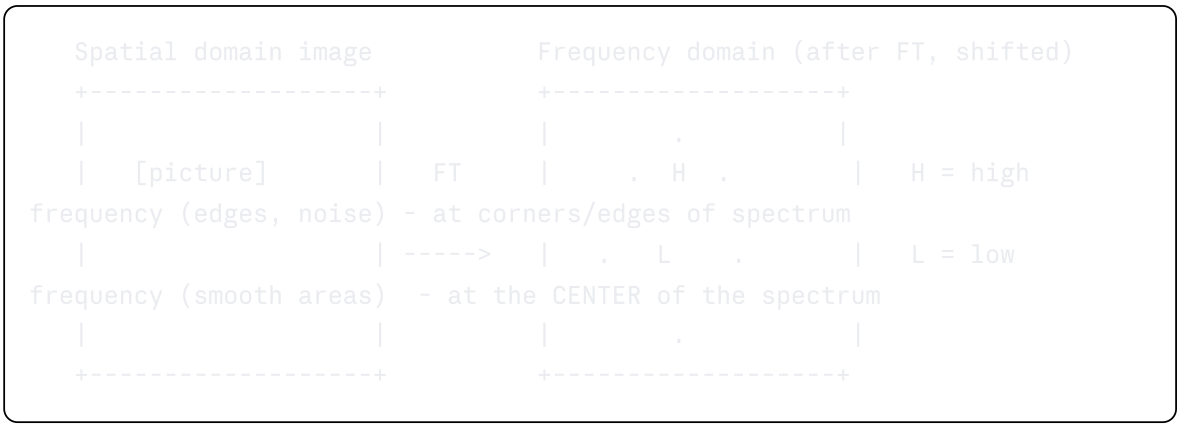
where f is the input image, T is some operator (e.g., a kernel convolution), and g is the output image.

4.2 Frequency Domain Filtering

> **Definition:** The image is first transformed into the **frequency domain** using the **Fourier Transform**, filtering is performed by modifying the frequency components (multiplying by a filter function $H(u, v)$), and then the result is converted back to the spatial domain using the **Inverse Fourier Transform**.

$$G(u, v) = F(u, v) \cdot H(u, v) \quad \xrightarrow{\text{Inverse FT}} \quad g(x, y)$$

Intuition: Instead of thinking of an image as pixels, think of it as a sum of many sine/cosine waves of different frequencies. Slowly-varying regions (sky, walls) = **low frequency**. Rapidly-changing regions (edges, texture, noise) = **high frequency**. Frequency domain filtering directly boosts or suppresses these frequency bands.



4.3 Detailed Comparison Table

Aspect	Spatial Domain	Frequency Domain
Operates on	Pixel intensities directly	Fourier-transformed image (frequency components)
Basic operation	Convolution with kernel	Multiplication with filter function $H(u, v)$
Mathematical basis	Convolution theorem (direct)	Fourier Transform, Convolution Theorem: Convolution in spatial = Multiplication in frequency
Computational cost	Cheap for small kernels; expensive for very large kernels	Expensive for small kernels (needs FT + inverse FT overhead), efficient for very large kernel sizes using FFT
Interpretability	Easy — clear physical meaning (neighborhood averaging etc.)	Harder — requires understanding of frequency concepts
Common uses	Smoothing, sharpening, edge detection (Sobel, Canny, etc.)	Periodic noise removal, global smoothing/sharpening, image restoration
Examples	Mean, Gaussian, Median, Sobel, Laplacian	Ideal, Butterworth, Gaussian frequency filters
Tools	`cv2.filter2D`, `cv2.blur`, `cv2.medianBlur`	`numpy.fft`, `cv2.dft`

> 📌 **Key Theorem — Convolution Theorem:** $$ f(x,y) * h(x,y) \iff F(u,v) \cdot H(u,v) $$ Convolution in the spatial domain is equivalent to **multiplication** in the frequency domain (and vice versa). This is why large-kernel spatial filtering can be sped up by transforming to frequency domain, multiplying, and transforming back.

4.4 When to Use Which?

- Use **spatial domain** when: kernel is small, operation is simple (blur, sharpen, edge detect), real-time performance needed.
- Use **frequency domain** when: need to remove **periodic/structured noise** (e.g., scan-line noise, moiré patterns), need very large-kernel smoothing efficiently, or need to analyze/globally understand image frequency content.

4.5 Common Mistakes

- ❌ Thinking frequency domain filtering is "always better" — it has overhead (needs FFT/IFFT) and is not intuitive for local operations.
- ❌ Forgetting to **shift the zero-frequency (DC) component to the center** of the spectrum before applying most textbook-defined filters (`np.fft.fftshift`).
- ❌ Confusing "frequency" in images with "frequency" in audio — image frequency refers to **rate of intensity change across space**, not time.

4.6 Chapter Summary

> Spatial domain filtering operates directly on pixels using kernels; frequency domain filtering operates on the Fourier-transformed image by multiplying with a filter function. They are mathematically linked by the **Convolution Theorem**. Spatial methods are simpler/faster for small kernels; frequency methods excel at large-scale/periodic-noise problems.

📋 **Quick Revision — Chapter 4**

- Spatial domain: direct pixel + kernel convolution.
- Frequency domain: FT → multiply by $H(u, v)$ → Inverse FT.
- Convolution Theorem: convolution (spatial) = multiplication (frequency).
- Frequency domain great for periodic noise removal & large-kernel filtering.
- Center of shifted FT spectrum = low frequencies; edges = high frequencies.

Chapter 5: Smoothing (Low-Pass) Filters

> **Definition:** Smoothing filters reduce sharp intensity variations (noise, fine detail) by averaging/combining neighboring pixel values, resulting in a **blurred** image. They act as **low-pass filters** — they let low frequencies (smooth regions) pass and suppress high frequencies (edges, noise).

5.1 Mean Filter (Averaging Filter)

Theory: Every pixel is replaced by the **arithmetic mean** of itself and its neighbors.

Kernel (3×3):

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Formula:

$$g(x, y) = \frac{1}{mn} \sum_{(s,t) \in \text{window}} f(x + s, y + t)$$

where $m \times n$ is the window/kernel size.

Advantages	Disadvantages
Simple, fast to compute	Blurs edges (loses detail)
Effective for Gaussian-type noise	Not effective for salt-and-pepper noise (spreads outlier values)
Easy to implement in hardware	Uniform blur — no edge preservation

Applications: Quick noise smoothing, pre-processing before edge detection, reducing high frequency noise in relatively flat regions.

Python/OpenCV:

```
python
```

```python
import cv2
img = cv2.imread('input.jpg', cv2.IMREAD_GRAYSCALE)
mean_filtered = cv2.blur(img, (3, 3))   # box averaging filter
cv2.imwrite('mean_filtered.jpg', mean_filtered)
```

5.2 Box Filter

Theory: A generalization of the mean filter where the **normalization is optional.** If normalized (divide by number of elements), it's identical to the mean filter. If un-normalized, it computes a simple sum (used internally for things like integral images).

Kernel: Same shape as mean filter, but can be normalized or not:

$$K_{\text{unnormalized}} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

> 📌 **Note Box:** Box filter = Mean filter when `normalize=True`. OpenCV keeps them as separate functions (`cv2.boxFilter` vs `cv2.blur`) but mathematically `cv2.blur()` is just `cv2.boxFilter()` with normalization forced on.

Python/OpenCV:

```python
box_filtered = cv2.boxFilter(img, -1, (5,5), normalize=True)
```

5.3 Gaussian Filter

Theory: Instead of giving every neighbor **equal weight** (like Mean filter), the Gaussian filter gives **higher weight to pixels closer to the center**, and progressively lower weight to farther pixels — following a **Gaussian (bell curve) distribution**. This produces a smoother, more natural blur that preserves edges better than the mean filter.

Formula (Continuous 2D Gaussian function):

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

σ (sigma) controls the spread — larger σ → more blur.

Example Kernel (5×5, $\sigma \approx 1$, approximate integer weights):

$$K = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

```
        Weight distribution (top view):
              low
           low  med  low
        low med HIGH med low
           low  med  low
              low
   (Center pixel gets maximum weight; weight
    decreases smoothly with distance — bell shape)
```

Advantages	Disadvantages
Smoother, more natural-looking blur than mean filter	Still blurs edges (though less than mean filter of same size)
Removes Gaussian noise very effectively	Not ideal for salt-and-pepper (impulse) noise
Separable (can be applied as two 1D passes — faster)	Choosing correct σ requires tuning
Widely used as pre-processing (e.g., before Canny)	Larger kernel/σ → more computation

Applications: Standard blur in almost all CV pipelines, noise reduction before edge detection (Canny's first step!), reducing high-frequency detail before downsampling.

Python/OpenCV:

```python
gaussian_filtered = cv2.GaussianBlur(img, (5, 5), sigmaX=1.0)
```

5.4 Median Filter

Theory: Instead of averaging, replace each pixel with the **median** (middle value when sorted) of the neighborhood. This is a **non-linear** filter (unlike Mean/Gaussian which are linear/weighted-sum).

Working Example: For a 3×3 window:

```
Neighborhood:        Sorted values:
  10  200  12         [10, 11, 12, 13, 14, 15, 20, 200, 255]
  11   14  13                          ^
  20   15  255                  MEDIAN = 14
```

Output pixel = 14 (the noisy values 200 and 255 are **completely ignored**, unlike mean filter which would be dragged toward them).

Formula:

$$g(x, y) = \text{median}\{f(s, t) \mid (s, t) \in \text{neighborhood of } (x, y)\}$$

Advantages	Disadvantages
Excellent at removing **salt-and-pepper (impulse) noise**	Slower than mean/Gaussian (needs sorting)
Preserves edges much better than mean filter	Not linear — harder to analyze mathematically
Doesn't introduce new pixel values (output is always an existing pixel value from the neighborhood)	Can remove thin lines/small details if window too large

Applications: Removing salt-and-pepper noise, pre-processing for OCR/document scanning, medical image denoising.

Python/OpenCV:

```python
median_filtered = cv2.medianBlur(img, 3)   # kernel size must be odd
```

5.5 Bilateral Filter

Theory: A Gaussian filter blurs everything equally regardless of whether it's crossing an edge. The **Bilateral filter** improves this by considering **two** Gaussian weights simultaneously:

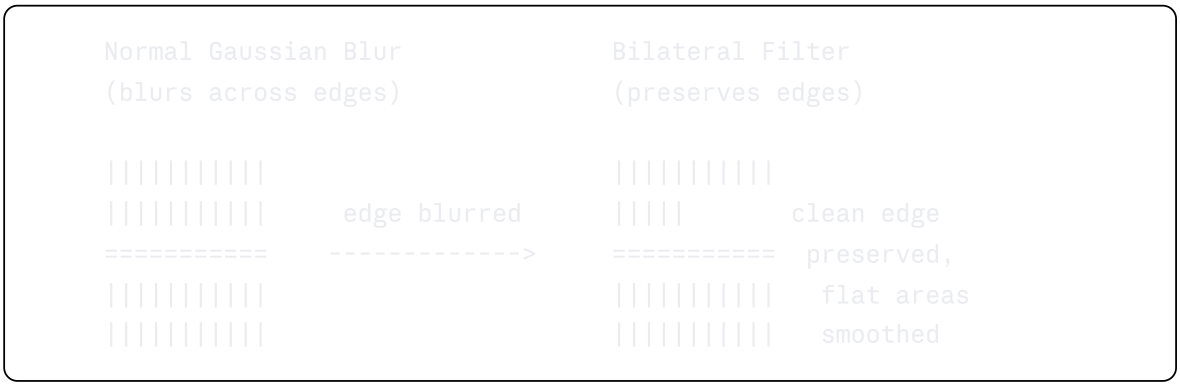
1. **Spatial closeness** (like normal Gaussian — nearby pixels matter more).
2. **Intensity similarity** (pixels with similar intensity to the center matter more, even if slightly farther away).

This way, it **smooths flat regions** but **preserves edges** (because pixels across an edge have very different intensity, so they get low weight).

Formula:

$$g(x,y) = \frac{1}{W_p} \sum_{(s,t)} f(s,t) \cdot \underbrace{e^{-\frac{(x-s)^2+(y-t)^2}{2\sigma_d^2}}}_{\text{spatial (domain) weight}} \cdot \underbrace{e^{-\frac{(f(x,y)-f(s,t))^2}{2\sigma_r^2}}}_{\text{intensity (range) weight}}$$

where W_p is a normalization constant (sum of all weights), σ_d = spatial sigma, σ_r = range/intensity sigma.



Advantages	Disadvantages
Preserves edges while smoothing flat/noisy regions	Computationally expensive (much slower than Gaussian)
Great for beautification (skin smoothing) & denoising while keeping structure	Two parameters to tune (σ_d, σ_r) — harder to configure
Widely used in photography apps & HDR tone mapping	Not separable like Gaussian (in its original form) — slower

Applications: Skin smoothing in photo editing, edge-preserving denoising, HDR tone mapping, cartoonification effects.

Python/OpenCV:

```python
bilateral_filtered = cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=
```

5.6 Comparison Table — Smoothing Filters

Filter	Type	Edge Preservation	Noise Best Suited For	Speed	Linear?
Mean/Box	Averaging	Poor	Gaussian noise	Fastest	Yes
Gaussian	Weighted averaging	Moderate	Gaussian noise	Fast (separable)	Yes
Median	Order statistic	Good	Salt-and-pepper noise	Moderate (sorting)	No
Bilateral	Weighted (space+range)	Excellent	Gaussian noise + preserving edges	Slowest	No

5.7 Common Mistakes

- ❌ Using Mean filter to remove salt-and-pepper noise (it spreads the noise instead of removing it — use Median instead).
- ❌ Forgetting kernel size must be **odd** for `cv2.medianBlur`/`GaussianBlur`.
- ❌ Using very large σ/kernel thinking it will "clean" the image — it destroys real detail too.
- ❌ Confusing Box filter (simple average, can be un-normalized) with Gaussian filter (weighted average, always normalized, bell-shaped weights).

5.8 Chapter Summary

> Smoothing filters are low-pass filters that reduce noise and detail via neighborhood combination. **Mean/Box**: fast, equal-weight, blurs edges. **Gaussian**: weighted by distance, smoother blur. **Median**: non-linear, best for impulse noise, strong edge-preservation. **Bilateral**: weighted by both distance and intensity similarity, best edge preservation but slowest.

📋 **Quick Revision — Chapter 5**

- Mean: equal weights, fast, blurs edges, good for Gaussian noise.
- Gaussian: bell-curve weights, smoother blur, separable, standard pre-processing.
- Median: non-linear, sorts neighborhood, picks middle value, best for salt-and-pepper noise, strong edge preservation.
- Bilateral: spatial + intensity weighting, best edge preservation, but slow.
- OpenCV: `cv2.blur`, `cv2.boxFilter`, `cv2.GaussianBlur`, `cv2.medianBlur`, `cv2.bilateralFilter`.

Chapter 6: Sharpening (High-Pass) Filters

6.1 Concept

> **Definition:** Sharpening filters **enhance edges and fine details** by emphasizing regions of rapid intensity change (high frequencies) and often **de-emphasizing** smooth regions. They are the opposite goal of smoothing filters.

Core idea: If we can compute "how blurred" an image is (using a low-pass version), we can **subtract the blur** or **add back the high-frequency detail** to make the image appear sharper.

6.2 High-Pass Filtering

A high-pass filter keeps high frequencies (edges/detail) and suppresses low frequencies (smooth regions). A simple relation:

$$\text{High-pass result} = \text{Original} - \text{Low-pass (blurred) version}$$

This is exactly the **basis of Unsharp Masking** (Section 6.4).

6.3 Laplacian Filter (as a Sharpening Tool)

Theory: The Laplacian is a **second-order derivative** operator that highlights regions of rapid intensity change in all directions (it is **isotropic** — rotation invariant). Since it responds strongly at edges, adding the Laplacian back to the original image sharpens it.

Formula (continuous):

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Common discrete Laplacian kernels:

$$K_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \qquad K_2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

(K_1 considers only 4-connected neighbors; K_2 considers all 8 neighbors — a stronger response.)

Sharpening using Laplacian:

$$g(x, y) = f(x, y) - c \cdot \nabla^2 f(x, y)$$

(The sign of c depends on the kernel's center sign convention — with the kernel above (center negative), we typically **subtract** the Laplacian result to sharpen. Many textbooks

instead directly define a "sharpening kernel" that already combines original + Laplacian, e.g.:)

$$K_{\text{sharpen}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

This single kernel simultaneously keeps the original image (the "5" at the center accounts for original + Laplacian contribution) and adds edge emphasis.

6.4 Unsharp Masking

Theory: A classic photographic technique adapted to digital images. Steps:

1. **Blur** the original image (Gaussian blur) → this is the "mask".
2. **Subtract** the blurred version from the original → gives the high-frequency detail ("edge mask").
3. **Add** this detail back to the original, scaled by an amount k.

Formula:

$$g(x,y) = f(x,y) + k\big[f(x,y) - \bar{f}(x,y)\big]$$

where $\bar{f}$ is the Gaussian-blurred version of f, and $k > 0$ controls sharpening strength.



6.5 High Boost Filtering

Theory: A generalization of unsharp masking where the original image is **amplified** ($A > 1$) before subtracting the blurred version — giving control over how much of the original vs. how much extra sharpening detail is retained.

Formula:

$$g(x,y) = A \cdot f(x,y) - \bar{f}(x,y) = (A-1)f(x,y) + \big[f(x,y) - \bar{f}(x,y)\big]$$

- If $A = 1$ → reduces to standard **unsharp masking**.
- If $A > 1$ → **High-boost filtering**, keeps more of the original low-frequency content while still adding the high-frequency (edge) detail — used when you don't want to lose brightness/contrast information.

6.6 Comparison — Sharpening Techniques

Technique	Core Idea	Formula	Use Case
Laplacian sharpening	Add 2nd derivative	$f - c\nabla^2 f$	Direct, simple edge emphasis
Unsharp Masking	Add (original − blurred)	$f + k(f - \bar{f})$	Photography, general sharpening
High-Boost Filtering	Amplify original + add detail	$Af - \bar{f}$	When retaining brightness matters (e.g., medical images)

6.7 Advantages, Disadvantages, Applications

Advantages	Disadvantages
Enhances edges/details, improves perceived quality	Amplifies noise as well as edges (sharpening a noisy image = noisy + sharp artifacts)
Useful pre-processing before feature extraction	Can create ringing/halo artifacts near strong edges
Simple to implement with kernels	Over-sharpening looks unnatural

Applications: Photography (detail enhancement), medical imaging (organ boundary clarity), satellite image enhancement, restoring slightly blurred/out-of-focus images, pre-processing before OCR.

6.8 Python/OpenCV Implementation

```python
```

```python
import cv2
import numpy as np

img = cv2.imread('input.jpg', cv2.IMREAD_GRAYSCALE).astype(np.float32)

# --- 1. Laplacian Sharpening ---
laplacian_kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]], dtype=np.float32)
sharpened_laplacian = cv2.filter2D(img, -1, laplacian_kernel)

# --- 2. Unsharp Masking ---
blurred = cv2.GaussianBlur(img, (5,5), sigmaX=2)
k = 1.5
unsharp = cv2.addWeighted(img, 1 + k, blurred, -k, 0)

# --- 3. High-Boost Filtering ---
A = 1.8
high_boost = A * img - blurred   # equivalent form: (A-1)*img + (img - blurre

# Clip and convert back to uint8 for saving/display
sharpened_laplacian = np.clip(sharpened_laplacian, 0, 255).astype(np.uint8)
unsharp = np.clip(unsharp, 0, 255).astype(np.uint8)
high_boost = np.clip(high_boost, 0, 255).astype(np.uint8)

cv2.imwrite('sharpened_laplacian.jpg', sharpened_laplacian)
cv2.imwrite('unsharp_masked.jpg', unsharp)
cv2.imwrite('high_boost.jpg', high_boost)
```

6.9 Common Mistakes

- ❌ Forgetting to **clip pixel values** to [0, 255] after sharpening (values can overflow/underflow).
- ❌ Applying sharpening on a **noisy** image without denoising first — this amplifies noise badly.
- ❌ Confusing Unsharp Masking's k with High-Boost's A — they play different roles (k scales just the *detail*, A scales the *entire original* before subtracting blur).
- ❌ Using too strong a sharpening kernel/parameter — creates unnatural "halo" edges (ringing artifact).

6.10 Chapter Summary

> Sharpening filters emphasize high-frequency content (edges/detail). The **Laplacian** kernel directly measures second-order change; **Unsharp Masking** adds back (original − blurred) detail; **High-Boost Filtering** generalizes this by amplifying the original before subtracting the blur, useful when brightness must be preserved.

📋 Quick Revision — Chapter 6

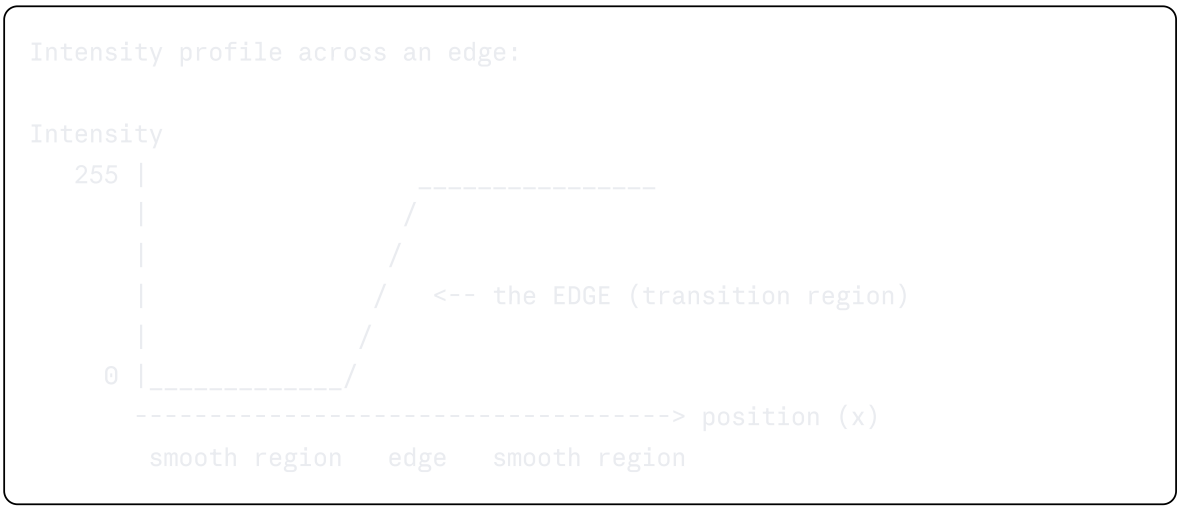
- Sharpening = boosting high frequencies (opposite of smoothing).
- Laplacian: 2nd derivative, isotropic, kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ (self-contained sharpening version).

- Unsharp Masking: $f + k(f - \bar{f})$.
- High-Boost: $Af - \bar{f}, A > 1$ generalizes unsharp masking.
- Risk: noise amplification, ringing artifacts, need clipping.

Chapter 7: Edge Detection Fundamentals

7.1 What Are Edges?

> **Definition:** An **edge** is a location in an image where there is a **significant/abrupt change in pixel intensity** — typically marking the boundary between two different regions (objects, surfaces, textures).



Edges usually correspond to:

- Object boundaries
- Changes in surface orientation (depth discontinuity)
- Changes in material/texture
- Shadows/illumination changes

7.2 Gradient — The Core Mathematical Tool

> **Definition:** The **image gradient** is a vector that points in the direction of the greatest rate of increase of intensity, with magnitude equal to the rate of change.

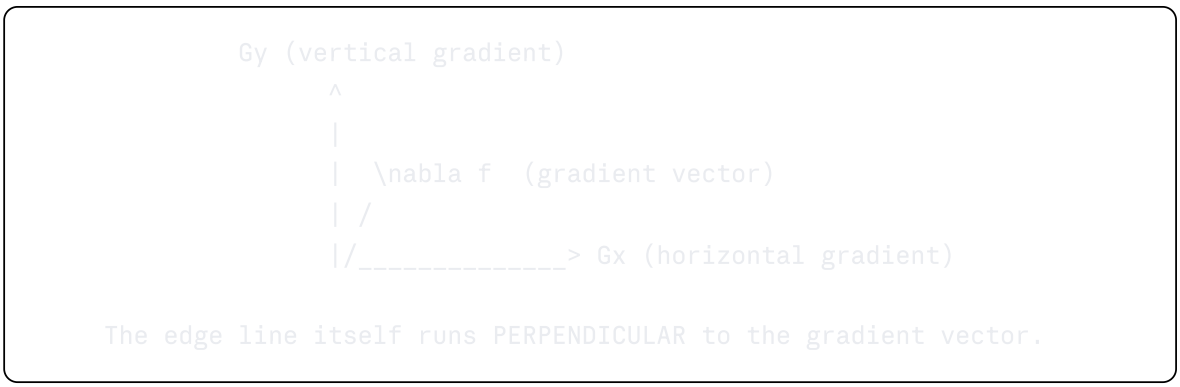
$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Gradient Magnitude (how strong the edge is):

$$|\nabla f| = \sqrt{G_x^2 + G_y^2} \quad (\text{often approximated as } |G_x| + |G_y| \text{ for speed})$$

Gradient Direction (orientation of the edge, perpendicular to the edge itself):

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$



7.3 First-Order Derivative Edge Detection

Theory: Approximates $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ using finite differences between neighboring pixels. Used by **Sobel, Prewitt, Roberts** operators (Chapters 8–10).

Simplest approximation:

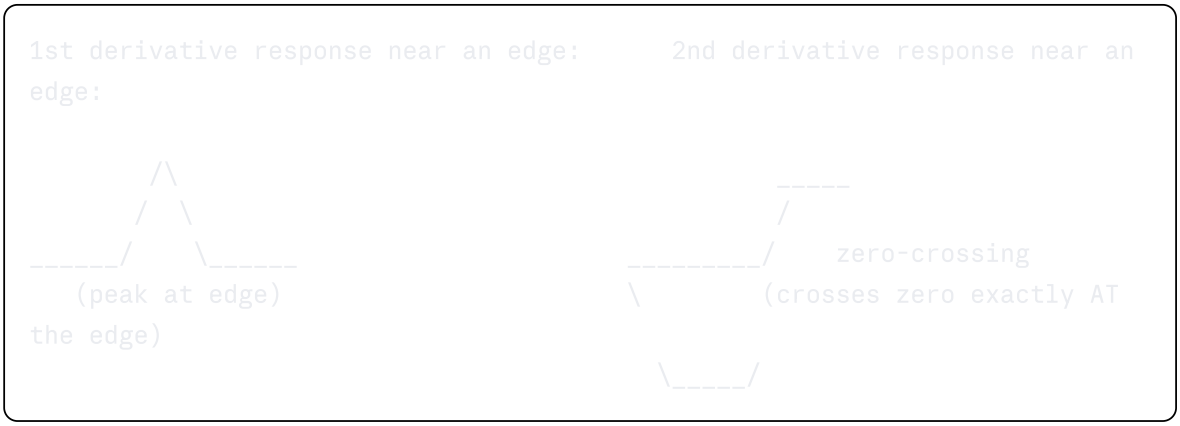
$$\frac{\partial f}{\partial x} \approx f(x+1, y) - f(x, y)$$

Characteristics of first-order derivative edge detectors:

- Produce a **thick** edge response (a "ramp" gives a broad response).
- Simple, fast to compute.
- Sensitive to noise (derivatives amplify high-frequency noise).

7.4 Second-Order Derivative Edge Detection

Theory: Uses the second derivative ($\frac{\partial^2 f}{\partial x^2}$), which is **zero** in flat regions, **positive** on one side of an edge, **negative** on the other side, and crosses **zero exactly at the edge location** — called a **zero crossing**. Used by the **Laplacian** operator (Chapter 11) and forms the basis of **Laplacian of Gaussian (LoG)**.



Property	First-Order Derivative	Second-Order Derivative
Response at edge	Peak (local maximum)	Zero-crossing
Response in flat region	Zero	Zero
Edge localization	Thick edges (broad peak)	Thinner, more precise (zero crossing is a single point)
Noise sensitivity	High	Very high (double differentiation amplifies noise more)
Example operators	Sobel, Prewitt, Roberts	Laplacian, LoG

7.5 Common Mistakes

- ❌ Thinking gradient magnitude alone tells you the edge's orientation — you need the **direction/angle** formula too.
- ❌ Forgetting that derivative-based methods are **very sensitive to noise** — that's why smoothing (Gaussian blur) is applied *before* edge detection in practice (see Canny, Chapter 12).
- ❌ Using $\sqrt{G_x^2 + G_y^2}$ and $|G_x| + |G_y|$ interchangeably without realizing the second is only an approximation (faster but less accurate).

7.6 Chapter Summary

> Edges are abrupt intensity transitions. They are detected using the **image gradient** — first-order derivatives give peak responses at edges (Sobel/Prewitt/Roberts), while second-order derivatives give zero-crossings at edges (Laplacian). Gradient magnitude measures edge strength; gradient direction gives edge orientation.

📋 **Quick Revision — Chapter 7**

- Edge = abrupt intensity change, marks boundaries.
- Gradient vector (G_x, G_y); magnitude $\sqrt{G_x^2 + G_y^2}$; direction $\tan^{-1}(G_y/G_x)$.
- First-order derivative → peak response at edges (thick edge).
- Second-order derivative → zero-crossing at edges (thin, precise, more noise-sensitive).

Chapter 8: Sobel Operator

8.1 Theory

> **Definition:** The Sobel operator is a **first-order derivative-based** edge detector that computes an approximation of the gradient using two 3×3 kernels — one for horizontal changes, one for vertical changes. It applies a small amount of **implicit smoothing** (via weighting) making it less noise-sensitive than a plain derivative.

8.2 Horizontal Kernel (G_x — detects vertical edges)

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

8.3 Vertical Kernel (G_y — detects horizontal edges)

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Intuition: G_x compares left vs. right columns (weighted more toward the center row) → strong response where intensity changes **horizontally** (i.e., detects **vertical** edges). Similarly G_y compares top vs. bottom rows → detects **horizontal** edges. The extra weight "2" at the center row/column (compared to Prewitt's uniform "1"s) gives a small smoothing effect, reducing noise sensitivity.

8.4 Gradient Magnitude and Direction

$$|G| = \sqrt{G_x^2 + G_y^2} \qquad (\text{or approx. } |G_x| + |G_y|)$$

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

8.5 Worked Example

Given image patch:

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 100 \\ 10 & 10 & 100 \end{bmatrix}$$

$$G_x = (-1)(10) + (0)(10) + (1)(10) + (-2)(10) + (0)(10) + (2)(100) + (-1)(10) + (0)(10) + (1$$

$$= (-10 + 0 + 10) + (-20 + 0 + 200) + (-10 + 0 + 100) = 0 + 180 + 90 = 270$$

$$G_y = (-1)(10) + (-2)(10) + (-1)(10) + 0 + 0 + 0 + (1)(10) + (2)(10) + (1)(100)$$

$$= (-10 - 20 - 10) + 0 + (10 + 20 + 100) = -40 + 130 = 90$$

$$|G| = \sqrt{270^2 + 90^2} = \sqrt{72900 + 8100} = \sqrt{81000} \approx 284.6$$

A large gradient magnitude → strong edge detected at this location (as expected, since intensity jumps from 10 to 100).

8.6 Advantages and Limitations

Advantages	Limitations
Simple, fast, easy to implement	Sensitive to noise (though less than Prewitt/Roberts due to weighting)
Provides both magnitude and direction	Produces thick edges (needs thinning, e.g., non-max suppression)
Some inherent smoothing effect	Not as accurate as Canny for precise edge localization
Widely supported in libraries (OpenCV, MATLAB)	Doesn't include hysteresis/thresholding logic on its own

8.7 Python/OpenCV Code

```python
import cv2
import numpy as np

img = cv2.imread('input.jpg', cv2.IMREAD_GRAYSCALE)

sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)   # G_x
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)   # G_y

magnitude = cv2.magnitude(sobel_x, sobel_y)
magnitude = cv2.convertScaleAbs(magnitude)   # convert back to 0-255 uint8

cv2.imwrite('sobel_edges.jpg', magnitude)
```

8.8 Common Mistakes

- ❌ Using `cv2.CV_8U` as the output depth for Sobel — this **clips negative gradients to 0**, losing half the edge information. Always use `cv2.CV_64F` (or `CV_16S`) then take absolute value.
- ❌ Forgetting to combine G_x and G_y into a final magnitude image.
- ❌ Confusing Sobel's kernel weights (with the "2") with Prewitt's (all "1"s) — a very common mix-up in exams!

8.9 Chapter Summary

> Sobel operator computes horizontal (G_x) and vertical (G_y) gradients using weighted 3×3 kernels, combining them into gradient magnitude and direction. Its center-row/column weight of 2 gives mild noise-smoothing compared to Prewitt.

📋 Quick Revision — Chapter 8

- $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$
- $|G| = \sqrt{G_x^2 + G_y^2}, \theta = \tan^{-1}(G_y/G_x)$
- Mild built-in smoothing (weight 2) reduces noise sensitivity vs. Prewitt.

- OpenCV: `cv2.Sobel()`.

Chapter 9: Prewitt Operator

9.1 Working

> **Definition:** The Prewitt operator is very similar to Sobel — a first-order gradient-based edge detector using two 3×3 kernels — but uses **equal weighting (all 1s)** instead of Sobel's center-weighted (2s) kernels, meaning **no extra emphasis** on the center row/column.

Horizontal Kernel (G_x):

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -1 & 0 & +1 \\ -1 & 0 & +1 \end{bmatrix}$$

Vertical Kernel (G_y):

$$G_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{bmatrix}$$

Gradient magnitude/direction formulas are identical to Sobel's.

9.2 Comparison with Sobel

Aspect	Sobel	Prewitt
Center row/column weight	2 (extra emphasis)	1 (uniform)
Noise sensitivity	Slightly lower (due to weighting/smoothing)	Slightly higher
Edge sharpness	Marginally smoother/thicker response	Marginally sharper but noisier
Computational cost	Same (3×3 kernel)	Same (3×3 kernel)
Common use	Preferred choice in most modern CV libraries as default	Used in classical/theoretical treatments, simpler to derive

> 📌 **Exam Tip:** If asked to differentiate Sobel from Prewitt, always mention the **weighting difference** (2 vs 1) — this is the most commonly tested distinction.

9.3 Python/OpenCV Code

OpenCV doesn't provide a direct built-in Prewitt function; use `cv2.filter2D`:

```
python
```

```python
import cv2
import numpy as np

img = cv2.imread('input.jpg', cv2.IMREAD_GRAYSCALE).astype(np.float32)

prewitt_x_kernel = np.array([[-1,0,1],[-1,0,1],[-1,0,1]], dtype=np.float32)
prewitt_y_kernel = np.array([[-1,-1,-1],[0,0,0],[1,1,1]], dtype=np.float32)

prewitt_x = cv2.filter2D(img, -1, prewitt_x_kernel)
prewitt_y = cv2.filter2D(img, -1, prewitt_y_kernel)

magnitude = cv2.magnitude(prewitt_x, prewitt_y)
magnitude = cv2.convertScaleAbs(magnitude)

cv2.imwrite('prewitt_edges.jpg', magnitude)
```

9.4 Chapter Summary

> Prewitt is nearly identical to Sobel but uses uniform (1,1,1) weighting instead of Sobel's center-weighted (1,2,1) rows/columns, making it slightly more noise-sensitive but conceptually simpler.

📋 **Quick Revision — Chapter 9**

- Prewitt kernels use all-1 weights (no center emphasis), unlike Sobel's 1-2-1 weighting.
- Slightly noisier response than Sobel, otherwise same math (magnitude/direction formulas).
- No direct OpenCV function — use `cv2.filter2D` with custom kernel.

Chapter 10: Roberts Operator (Roberts Cross-Gradient)

10.1 Working

> **Definition:** The Roberts Cross operator is one of the **earliest and simplest** edge detectors. It uses tiny **2×2 kernels** applied diagonally, computing gradients along the diagonal directions rather than horizontal/vertical.

Kernels:

$$G_x = \begin{bmatrix} +1 & 0 \\ 0 & -1 \end{bmatrix} \qquad G_y = \begin{bmatrix} 0 & +1 \\ -1 & 0 \end{bmatrix}$$

Gradient magnitude:

$$|G| = \sqrt{G_x^2 + G_y^2}$$

Intuition: Instead of comparing left-right or top-bottom neighbors (like Sobel/Prewitt), Roberts compares **diagonal** neighbor pairs — essentially a minimal 2-pixel difference along each diagonal.

```
Pixel layout:        Gx measures:          Gy measures:
 P1  P2                 P1 - P4               P2 - P3
 P3  P4               (main diagonal)       (anti-diagonal)
```

10.2 Advantages

- Extremely simple and **fast** (smallest possible kernel — only 4 multiplications total for both kernels).
- Good at detecting edges with **diagonal orientation**.
- Low computational/memory footprint — historically used in early, resource-constrained systems.

10.3 Disadvantages

- Very **sensitive to noise** (uses only 2×2 pixels — no averaging/smoothing at all).
- Poor at detecting edges that are purely horizontal or vertical (biased toward diagonals).
- Not rotationally symmetric — inconsistent response depending on edge orientation.
- Rarely used today except in teaching/historical context or extremely resource-limited embedded systems.

10.4 Comparison Snapshot

Operator	Kernel Size	Directions Sensed	Noise Robustness	Speed
Roberts	2×2	Diagonal	Lowest	Fastest
Prewitt	3×3	Horizontal & Vertical	Medium	Fast
Sobel	3×3 (weighted)	Horizontal & Vertical	Medium-High	Fast

10.5 Python/OpenCV Code

python

```python
import cv2
import numpy as np

img = cv2.imread('input.jpg', cv2.IMREAD_GRAYSCALE).astype(np.float32)

roberts_x = np.array([[1, 0], [0, -1]], dtype=np.float32)
roberts_y = np.array([[0, 1], [-1, 0]], dtype=np.float32)

gx = cv2.filter2D(img, -1, roberts_x)
gy = cv2.filter2D(img, -1, roberts_y)

magnitude = cv2.magnitude(gx, gy)
magnitude = cv2.convertScaleAbs(magnitude)

cv2.imwrite('roberts_edges.jpg', magnitude)
```

10.6 Chapter Summary

> The Roberts Cross operator uses tiny 2×2 diagonal kernels — the simplest and fastest classical edge detector, but highly noise-sensitive and biased toward diagonal edges. Mostly of historical/educational importance today.

📋 Quick Revision — Chapter 10

- Roberts uses 2×2 kernels: $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ and $\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$.
- Detects diagonal edges; fastest but noisiest of the classical operators.
- Rarely used in modern practice; important for exam/theory questions.
